



UNIVERSIDADE
FEDERAL DO CARIRI

Introdução ao OpenGL com C

Professora Dra. Luana Batista da Cruz
luana.batista@ufca.edu.br

Roteiro

01 Instalação

02 OpenGL

03 Hello World com OpenGL

01

Instalação

Windows

Linux

Possíveis erros de instalação

Diretivas de compilação

Instalação OpenGL no Windows

- 1) Baixar o **MinGW** (<https://www.mingw-w64.org> ou https://drive.google.com/drive/folders/1-rRkvCay8gRyIh5ZZNpYVvOQPz1Z_82q?usp=drive_link - se for Windows 10). É um ambiente de desenvolvimento que dá suporte ao compilador GCC em sistemas Windows
 - a) Instalar em C:\MinGW (<MINGW_HOME>)
 - b) Na instalação do MinGW, instalar os pacotes básicos “mingw32-base”, “mingw32-gcc-g++” e “msys-base”
 - c) Colocar os diretórios <MINGW_HOME>\bin e <MINGW_HOME>\msys\1.0\bin na variável de ambiente Path do Windows

Instalação OpenGL no Windows

- O OpenGL geralmente já está instalado no Windows e Linux, mas para programas gráficos interativos, é recomendável usar bibliotecas auxiliares como GLU e FreeGLUT
 - **OpenGL Utility (GLU)**
 - Desenvolvida como uma extensão padrão (já vem instalada) do OpenGL para oferecer uma interface mais simples e fácil de usar. Fornece funções adicionais, como transformações de matriz, mapeamento de textura, projeção, cálculo de normais, entre outras
 - **Free OpenGL Utility Toolkit (FreeGLUT)**
 - Derivado do GLUT, logo oferece todas as funcionalidades do GLUT original e adiciona recursos como suporte a tela cheia, multi-janelas, suporte a teclado e mouse em terceira dimensão, funções de desenho de superfície e contextos OpenGL avançados

Instalação OpenGL no Windows

- 2) Para instalar a biblioteca FreeGLUT no MinGW
 - a) Faça o download da biblioteca https://drive.google.com/drive/folders/1cQi8lQwe98hFjQcXezO6Hn2YaiFCaGWx?usp=share_link. Contém os arquivos de cabeçalho (headers) e as bibliotecas estáticas (libs) e dinâmicas (bin)
 - b) Descompacte o arquivo baixado no passo anterior
 - c) Copie os arquivos que estão em “include\GL” para “<MINGW_HOME>\include\GL”
 - d) Os arquivos da pasta “lib” para “<MINGW_HOME>\lib”
 - e) Os arquivos da pasta “bin” para “<MINGW_HOME>\bin”
- 3) Baixar e instalar o CodeBlocks (versão sem MinGW) (<http://www.codeblocks.org>), caso goste de usar uma IDE para compilar seus programas em C

Instalação OpenGL no Linux

- Instalar os seguintes pacotes (podem ter leves variações nos nomes, depende da distribuição Linux)
 - **freeglut**
 - **freeglut-devel**
 - make
 - gcc
- Esses pacotes irão trazer dependências adicionais, instale todas elas para funcionar corretamente
- Instale o CodeBlocks, caso goste de usar uma IDE para compilar seus programas em C

Possíveis erros de instalação

1) Falta de algumas DLLs do OpenGL. Para isso, verifique a existência dos arquivos DLL executando os seguintes comandos no Prompt de Comando

a) No **Windows**

- `dir C:\Windows\System32\opengl32.dll`
- `dir C:\Windows\System32\glu32.dll`
- Se o arquivo `opengl32.dll` e `glu32.dll` estiverem presente, isso indica que a biblioteca OpenGL está instalada no sistema

b) No **Linux**

- `ls /usr/lib/libGL*`

Possíveis erros de instalação

2) Verifique se foi instalado corretamente o compilador MinGW

- a) Verifique a presença do MinGW por meio do seguinte comando
 - `gcc --version`
 - Se o comando não for reconhecido, significa que o GCC não está instalado, ou os diretórios do MinGW não estão na variável de ambiente Path do sistema
- b) Confira o diretório onde o MinGW foi instalado. Normalmente, o MinGW é instalado em algo como `C:\MinGW` ou `C:\MinGW-w64`, dependendo da versão

Possíveis erros de instalação

- 3) Verifique a configuração do MinGW no CodeBlocks
 - a) Clique na aba “Setting” e em “Compiler”
 - b) Depois que abrir a janela, vá na aba “Toolchain executables” e verifique se o diretório de instalação do compilador está correto (local onde instalou)
 - c) Caso não esteja, atualize

Diretivas de compilação

- São comandos que são passados ao compilador durante o processo de compilação de um programa, para especificar quais bibliotecas devem ser vinculadas ao programa
- No geral, o OpenGL é independente de sistema operacional, mas as diretivas são diferentes se for Windows ou Linux
- As seguintes diretivas de compilação devem ser usadas no seu projeto OpenGL, dependendo do sistema operacional
 - Se for **Windows**, “-lopengl32 -lglu32 -lfreetype”
 - Se for **Linux**, “-lGL -lglut -lGLU”

02

OpenGL

Introdução

API OpenGL

Programação dirigida a eventos

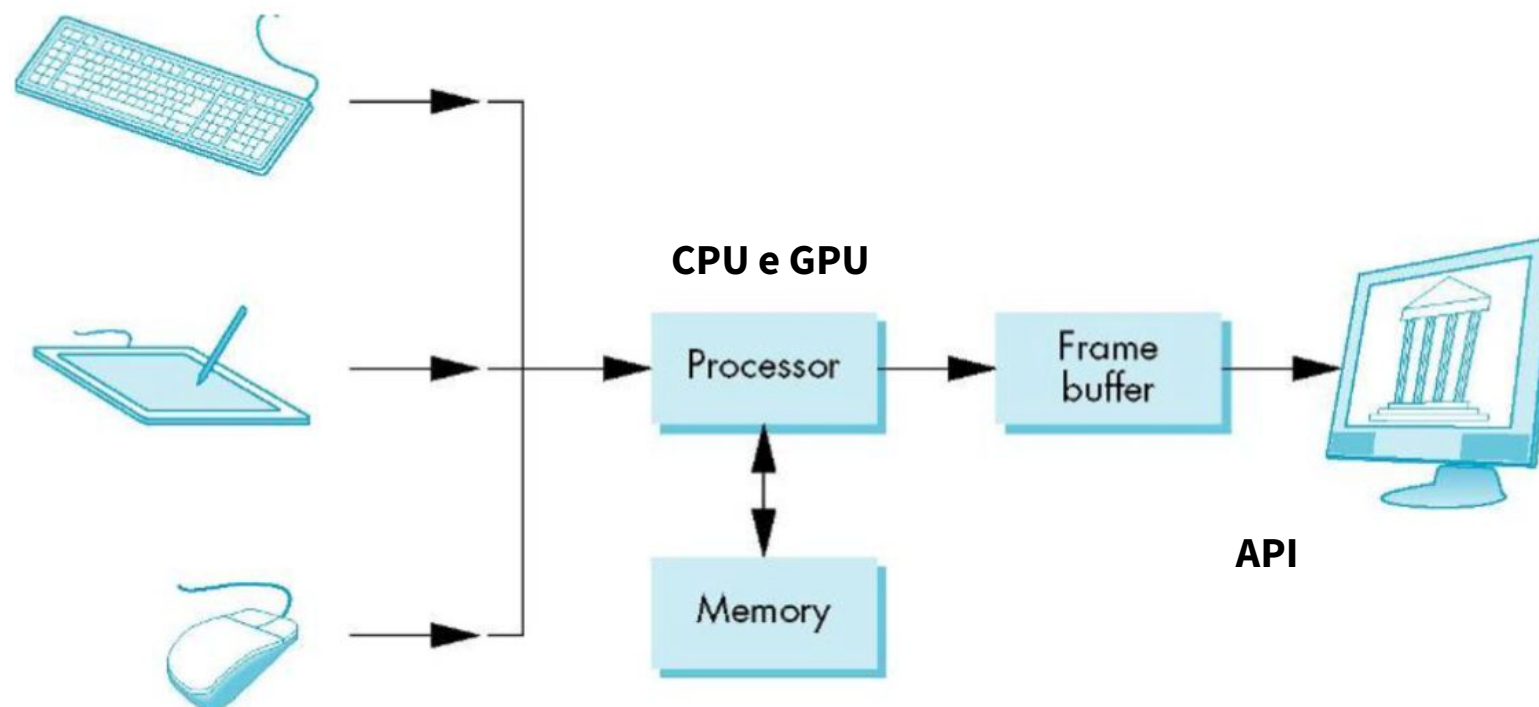
Estados do OpenGL

Primitivas

Programando - Linha vermelha

Introdução

- Normalmente para se trabalhar com computação gráfica precisamos de alguns hardware de entrada para interagir com os modelos



Introdução

- **Interface de Programação de Aplicativos (API)**

- Coleção de rotinas que o programador pode chamar (exemplo, OpenGL)
- Modelo de como estas rotinas operam em conjunto para gerar gráficos
- Programador “enxerga” apenas a interface
 - O OpenGL provê um protocolo de mais alto nível que encapsula rotinas de baixo nível que trabalha com o hardware. Por exemplo, dá uma instrução para desenhar uma linha (informa dois vértices) e o OpenGL preenche os pontos intermediários
- Não precisa lidar com aspectos específicos do hardware ou peculiaridades de software no sistema gráfico residente (independente da plataforma)
 - O mesmo código OpenGL pode ser usado em diferentes sistemas operacionais
- Oferece suporte para gerar e exibir cenas 3D complexas, e também para gráficos 2D simples

API OpenGL

- Ambiente para escrever e executar programas gráficos
 - Monitor + bibliotecas auxiliares para desenhar primitivas gráficas na tela
- API OpenGL pode ser vista como uma “**caixa preta**”
 - **Entradas:** chamadas as funções das bibliotecas por meio de dispositivos de entrada, etc
 - **Saídas:** os gráficos exibidos no monitor
 - Descrita em termos das funções que a API disponibiliza. Em resumo, o código C faz chamadas às bibliotecas do OpenGL, e a saída é uma janela que exhibe os gráficos com base nas entradas fornecidas

API OpenGL

- **Programa OpenGL**

- Em geral, trabalha com um sistema de janelas (window system)
- Inicializações: modo de exibição (display mode: frame buffer, modelo de cor), janela de desenho (janela de recorte) e sistema de coordenadas de referência (associado à janela)

- **API oferece centenas de funções...**

- Diferentes funcionalidades
 - **Funções primitivas:** o que vou desenhar?
 - **Funções de atributos:** que cor? Primitiva preenchida?
 - ...

Programação dirigida a eventos

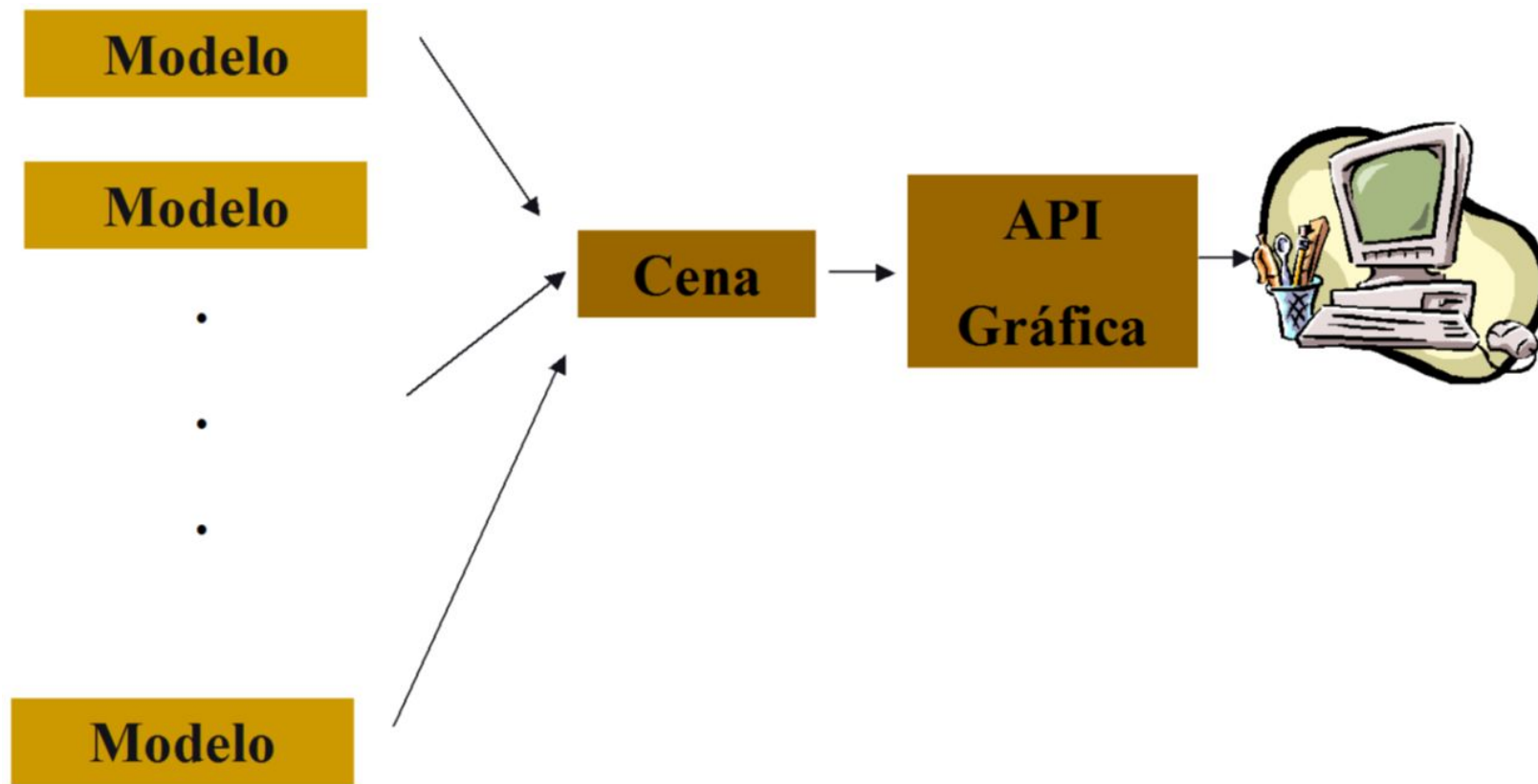
- **O OpenGL pode ser direcionada a eventos (event-driven)**
 - Programa responde a eventos: clique do mouse, tecla pressionada, redimensionamento da janela, ...

Programando

- OpenGL mantém diversas **variáveis de estado**, que são configurações que permanecem ativas até serem alteradas
- Exemplos de variáveis de estado
 - **Tamanho de ponto:** `glPointSize(3.0)`
 - **Cor de desenho:** `glColor3f(red, green, blue)`
 - **Cor de fundo da janela:** `glClearColor(red, green, blue, alpha)`
 - **Limpar janela:** `glClear(GL_COLOR_BUFFER_BIT)` – os bits do color buffer serão modificados para a cor de fundo definida em `glClearColor`

Programando

- Sistema gráfico



Programando – Linha vermelha

- O arquivo **glut.h** contém os protótipos das funções utilizadas pelo programa, e também inclui os headers **gl.h** e **glu.h** (OpenGL e GLU)

Informar para o arquivo C que vai ser um programa OpenGL

```
#include <windows.h>
#include <GL/glut.h>

int init(void) {
    /* coloca as funções de inicialização */
    /* inserir código */
}

void display(void) {
    /* coloca as funções do openGL referentes a desenho*/
    /* inserir código */
}

int main(int argc, char** argv) {
    /* inserir código */
}
```

Inclua o arquivo de cabeçalho windows.h, se estiver usando o Windows

Programando – Linha vermelha

- Precisamos de três funções principais
 - **Função init:** inicializa os parâmetros do rendering
 - **Função display:** será usado como função callback para fazer o desenho
 - **Função main:** função principal

```
#include <windows.h>
#include <GL/glut.h>

int init(void) {
    /* coloca as funções de inicialização */
    /* inserir código */
}

void display(void) {
    /* coloca as funções do openGL referentes a desenho*/
    /* inserir código */
}

int main(int argc, char** argv) {
    /* inserir código */
}
```

Programando – Linha vermelha

- Gerenciamento da janela de exibição usando GLUT

```
int main(int argc, char** argv) {  
  
    glutInit(&argc, argv);           //inicializa o GLUT  
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB); //configura o modo de display  
    glutInitWindowSize(400, 300);    //configura a largura e altura da janela de exibição  
    glutInitWindowPosition(200, 0);  
    glutCreateWindow("Um programa OpenGL Exemplo"); //cria a janela de exibição  
  
    init();                          //executa função de inicialização  
    glutDisplayFunc(display);        //estabelece a função "display" como a função callback de exibição  
    glutMainLoop();                 //mostre tudo e espere  
    return 0;  
}
```

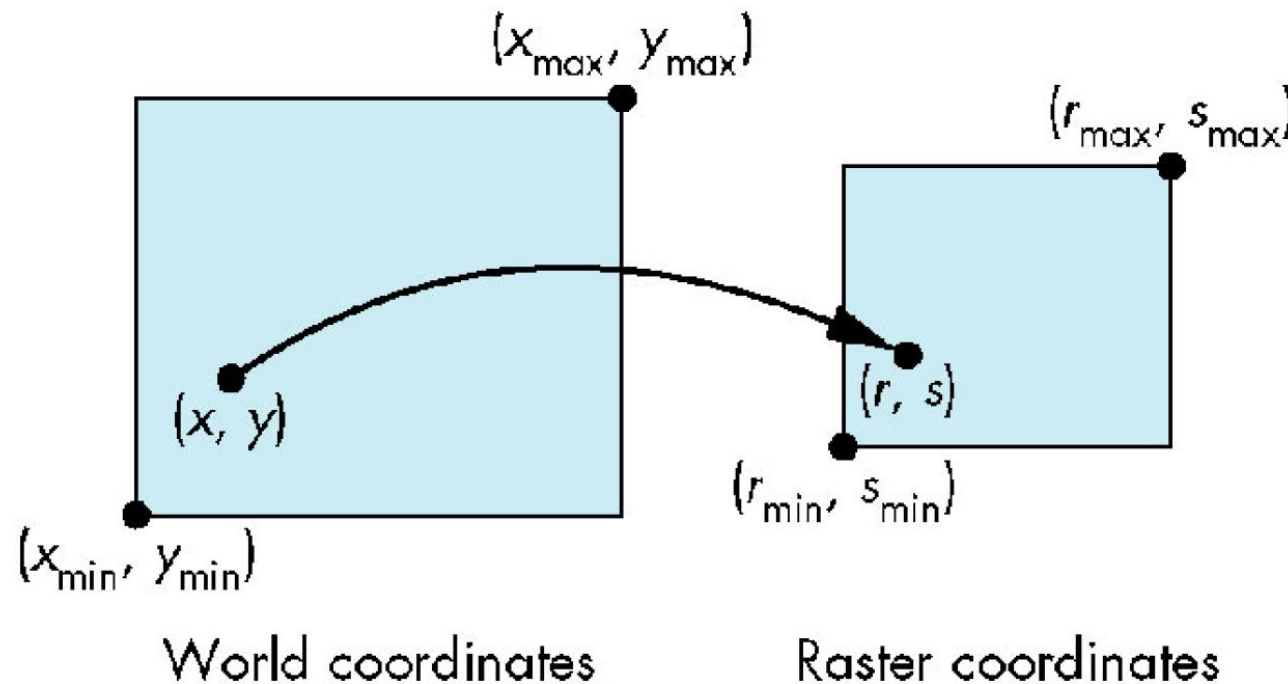
Programando – Linha vermelha

- Define a função de inicialização
 - **glMatrixMode:** modo de operação da matriz
 - **gluOrtho2D:** os parâmetros que estão sendo fornecidos trata-se da estrutura de janela de recorte. A área de desenho é como se tivesse no sistema cartesiano x, y (infinito). Portanto, é necessário selecionar qual parte do sistema cartesiano que vai parar na tela do openGL, setando o menor e o maior valor de x e y

```
int init(void){  
    glClearColor(1.0, 1.0, 1.0, 0.0); //define a cor de fundo  
  
    glMatrixMode(GL_PROJECTION); //carrega a matriz de projeção  
    gluOrtho2D(0.0, 200.0, 0.0, 150.0); //define projeção ortogonal 2D  
}
```

Programando – Linha vermelha

- O pipeline do OpenGL é sempre 3D, mas é possível criar desenhos 2D, definindo
 - `glMatrixMode(GL_PROJECTION)`
 - `gluOrtho2D(xw_min, xw_max, yw_min, yw_max)`



Programando – Linha vermelha

- Define a função de desenho

```
void display(void) {  
    glClear(GL_COLOR_BUFFER_BIT);    //desenha o fundo (limpa a janela)  
  
    glColor3f(1.0,0.0,0.0);         //altera o atributo de cor  
    glBegin(GL_LINES);              //desenha uma linha  
        glVertex2i(180,15);  
        glVertex2i(10,145);  
    glEnd();  
  
    glFlush();                       //desenha os comandos não executados  
}
```

Programando

- OpenGL permite desenhar gráficos de modo independente do dispositivo
- É possível especificar o Sistema de Coordenadas do Mundo (ponto flutuante) onde os objetos são definidos
- Os elementos são traçados no Sistema de Coordenadas do Dispositivo, ou Sistema de Coordenadas da Tela (inteiro)
 - O mapeamento (projeção bidimensional ou view 2D) entre esses sistemas de coordenadas é feito de forma transparente pelo OpenGL

Primitivas

- **Primitivas básicas**

- Pontos, linhas, polilinhas, polígonos (GL_POINTS, GL_LINES, GL_POLYGON, etc)
 - Polígonos são linhas fechadas formadas por segmentos de reta conectados que não se cruzam exceto nas extremidades, podendo ou não ser preenchidos
 - Polilinhas são segmentos de reta conectados de diferentes comprimentos e orientações. Podem ser aberta ou fechada, podem se cruzar, mas não são preenchidas
- Para descrever um objeto, uma lista de vértices é informada

```
glBegin(GL_POINTS);    //desenha 3 pontos
    glVertex2i(100, 50);
    glVertex2i(100, 130);
    glVertex2i(150, 130);
glEnd();
```

Primitivas

- As funções OpenGL possuem um formato que ajuda a identificá-las

Nome da função dimensões

`glVertex3f(x, y, z)`

pertence a biblioteca GL `x, y, z` são floats

`glVertex3fv(p)`

`p` é um ponteiro para um vetor

Outras primitivas: polilinhas e polígonos

- Polilinha: GL_LINE_STRIP, GL_LINE_LOOP
- Outras primitivas
 - GL_POLYGON
 - GL_TRIANGLES
 - GL_QUADS
 - GL_TRIANGLE_STRIP
 - GL_TRIANGLE_FAN
 - GL_QUAD_STRIP

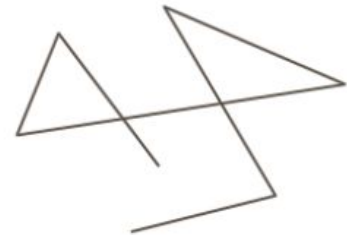
Outras primitivas: polilinhas e polígonos



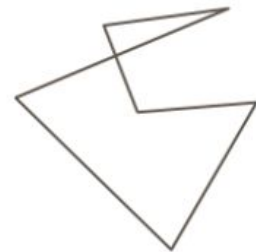
GL_POINTS



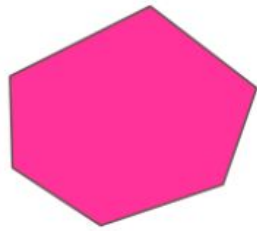
GL_LINES



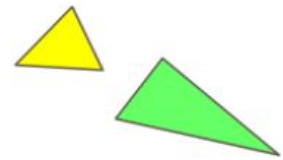
GL_LINE_STRIP



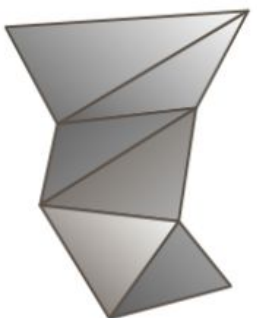
GL_LINE_LOOP



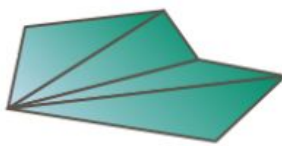
GL_POLYGON



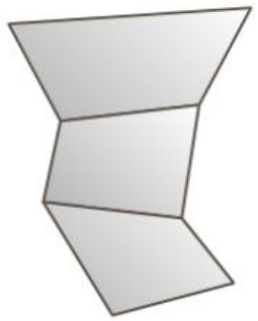
GL_TRIANGLES



GL_TRIANGLE_STRIP



GL_TRIANGLE_FAN



GL_QUAD_STRIP

Outras primitivas: polilinhas e polígonos

- GL_TRIANGLE_STRIP, GL_TRIANGLE_FAN E GL_QUAD_STRIP
 - São mais usados para estruturas tridimensionais, pois são boas para fazer malhas poligonais

Documentação do OpenGL

- <https://www.opengl.org/sdk/docs/man>
- <https://docs.gl/>

02

Hello World com OpenGL

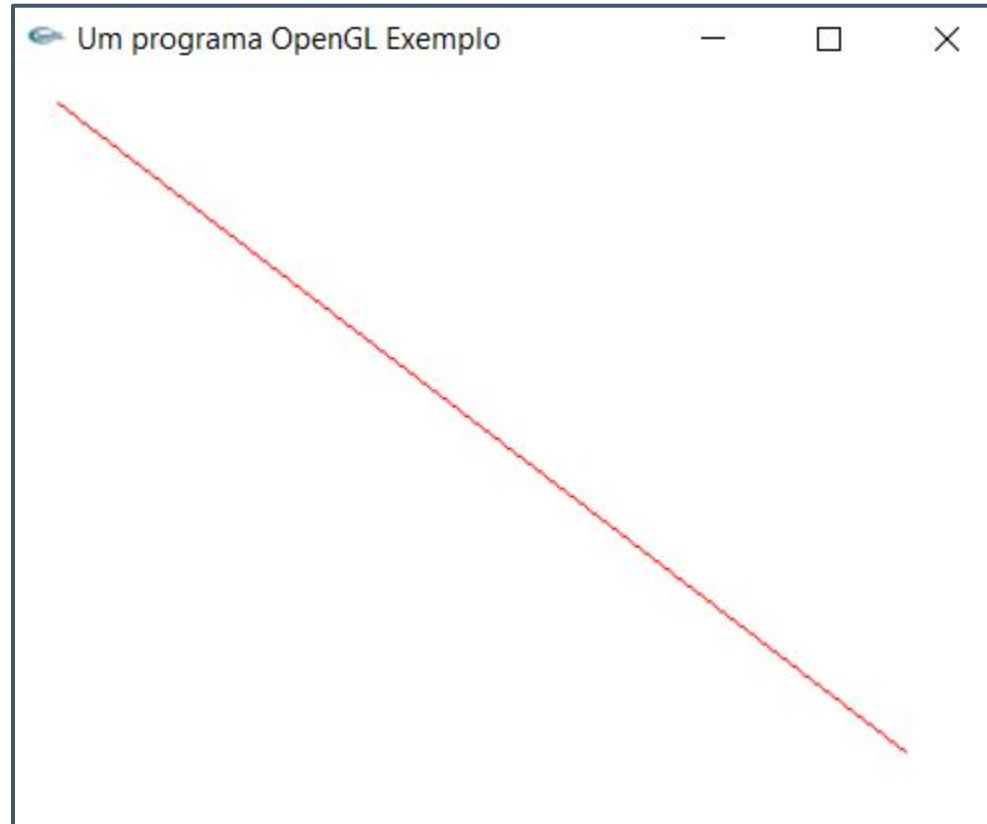
Testando sua instalação

Testando sua instalação

- Crie um projeto no CodeBlocks
- Implemente o arquivo “main.c” (versão windows ou versão linux)
- Clique com o botão direito do mouse no projeto depois em “Build Options...”. Na aba “Linker settings”, depois em “Other linker options” coloque o texto
 - Se for Windows, “-lopengl32 -lglu32 -lfreetype”
 - Se for Linux, “-lGL -lglut -lGLU”

Testando sua instalação

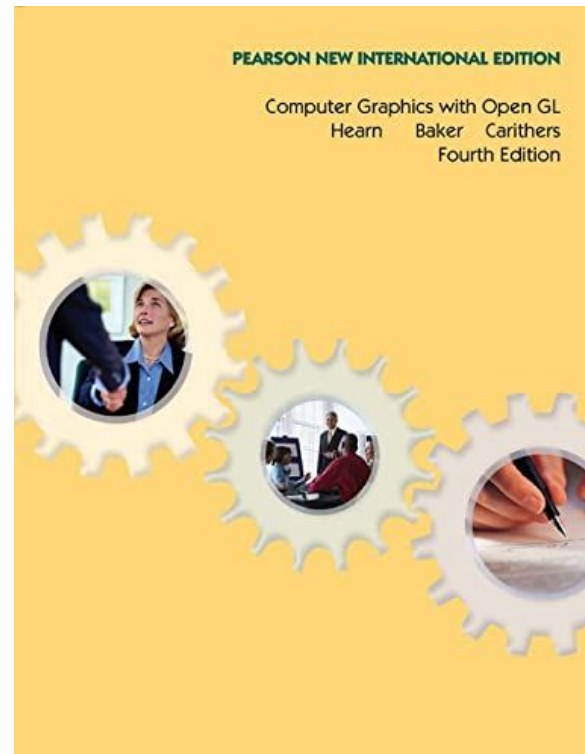
- Compile e execute o seu projeto
- Se tudo der certo, a seguinte janela deverá aparecer



Referências



AZEVEDO, Eduardo; CONCI, Aura.
Computação gráfica: teoria e prática.
Elsevier, 2003.



HEARN, DONALD. **Computer graphics with OpenGL.** Pearson Education Limited, 2014.
5º Edição.



PICHETTI, R. F; et al. **Computação Gráfica e Processamento de Imagens.**
Porto Alegre: SAGAH, 2022.

Referências



- ALENCAR, Aretha. **Notas de aula da disciplina Computação Gráfica da Universidade Tecnológica Federal do Paraná.** 2021.
- PAIVA, Anselmo. **Notas de aula da disciplina Computação Gráfica da Universidade Federal do Maranhão.** 2018.
- QUINTANILHA, Darlan. **Notas de aula da disciplina Computação Gráfica da Universidade Federal do Maranhão.** 2022.
- SILVA, Giovanni. **Notas de aula da disciplina Computação Gráfica da Universidade Federal do Maranhão.** 2019.